

Sprint 4.5 - API & Interfaces (NEXT)

Goal: Wire AEON's LLM components to actual user interfaces

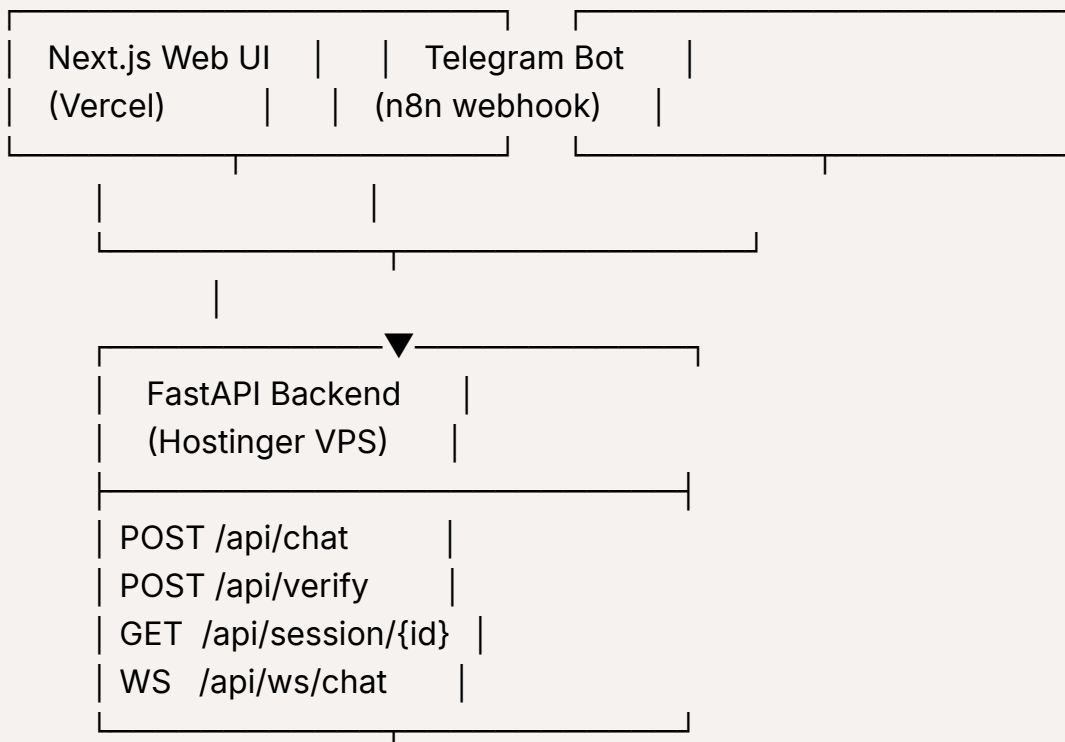
Rationale: Sprint 4 built the LLM integration; Sprint 4.5 makes it accessible to real users.

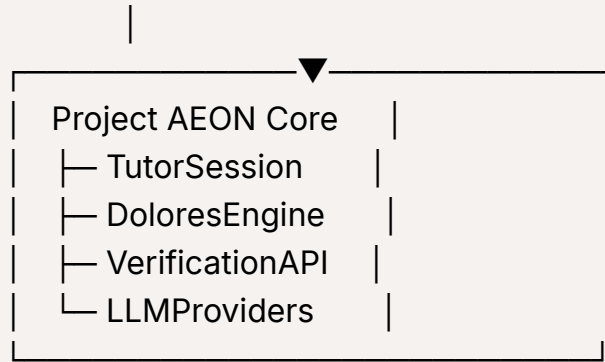
Deployment Targets:

- Web chat interface (Next.js on Vercel)
- Telegram bot with Dolores persona (replacing Doctor Feynman)
- Full stack: Verification + Persona + Future ZK Proofs

Status: Components are 90% API-ready, just need HTTP wrappers

Architecture





Implementation Phases

Phase 1: FastAPI Backend HIGH PRIORITY

Foundation for everything else

Files to Create:

`src/api/__init__.py` (~20 lines)

- Module exports

`src/api/app.py` (~100 lines)

- FastAPI application setup
- CORS configuration
- Lifespan management

`src/api/models.py` (~150 lines)

- Pydantic request/response models

```
class ChatRequest(BaseModel):
    message: str
    session_id: Optional[str]
    mode: Optional[str] # rigorous/exploratory/hybrid

class ChatResponse(BaseModel):
    response: str
    session_id: str
```

```
verified_claims: List[VerificationResultModel]
protocol_state: str
fidelity: float
```

`src/api/routes/ chat.py` (~150 lines)

- `POST /api/chat` - Send message, get response
- `GET /api/chat/stream` - SSE streaming response
- `WebSocket /api/ws/chat` - Real-time chat

`src/api/routes/ verify.py` (~100 lines)

- `POST /api/verify` - Verify a claim
- `GET /api/verify/gate/{name}` - Gate verification
- `GET /api/verify/state/{expr}` - State verification

`src/api/routes/ session.py` (~100 lines)

- `POST /api/session` - Create session
- `GET /api/session/{id}` - Get status
- `DELETE /api/session/{id}` - End session
- `POST /api/session/{id}/save` - Save capsule
- `POST /api/session/restore/{capsule_id}` - Restore

`src/api/session_ manager.py` (~150 lines)

- In-memory session store (upgrade to Redis later)
- Session lifecycle management
- Cleanup of expired sessions

`tests/test_ api.py` (~200 lines)

- API endpoint tests

Dependencies:

```
fastapi = ">=0.109.0"
uvicorn = ">=0.27.0"
```

```
python-multipart = ">=0.0.6"
```

Deployment:

- Run on existing Hostinger VPS alongside n8n
- `uvicorn src.api.app :app --host 0.0.0.0 --port 8000`
- Use nginx reverse proxy for HTTPS

Phase 2: Next.js Web UI HIGH PRIORITY

Primary user interface

Structure:

```
aeon-web/
├── app/
│   ├── page.tsx      # Landing/chat page
│   ├── layout.tsx    # Root layout
│   ├── api/          # API route proxies (optional)
│   └── chat/
│       └── page.tsx   # Main chat interface
├── components/
│   ├── Chat/
│   │   ├── ChatWindow.tsx # Main chat container
│   │   ├── MessageBubble.tsx # Individual messages
│   │   ├── VerificationBadge.tsx # Verified/unverified indicator
│   │   └── ModeSelector.tsx # Mode switching
│   ├── Dolores/
│   │   ├── StateIndicator.tsx # Protocol state display
│   │   └── FidelityMeter.tsx # Fidelity visualization
│   └── Verification/
│       ├── ProofDisplay.tsx # Show symbolic proof
│       └── ClaimList.tsx    # Verified claims list
├── lib/
│   ├── api.ts         # API client
│   └── types.ts        # TypeScript types
```

```
|— styles/
|   |— globals.css      # Tailwind + custom styles
|— public/
|   |— dolores-avatar.png  # Persona avatar
```

Key Features:

- Real-time chat with streaming responses
- Verification status badges on claims
- Mode toggle (Rigorous/Exploratory/Hybrid)
- Protocol state indicator
- Fidelity meter
- Dark mode (matching your n8n aesthetic)

Tech Stack:

- Next.js 14 (App Router)
- Tailwind CSS
- shadcn/ui components
- Vercel deployment

Phase 3: Telegram Integration 🟡 MEDIUM PRIORITY

Replace Doctor Feynman

Approach: n8n Webhook (Recommended)

Use existing n8n infrastructure:

Telegram Trigger → HTTP Request → FastAPI → Response → Telegram Send

n8n Workflow:

1. Telegram Trigger (message received)
2. Extract `user_id`, message text
3. HTTP Request to `POST /api/chat`

- Body: `{ message, session_id: user_id }`

4. Parse JSON response

5. Telegram Send Message (`response.response`)

Pros:

- Uses existing n8n setup
- No new deployment needed
- Easy to modify

Session Persistence:

- Use Telegram `user_id` as session identifier
- Store `capsule_id` in session manager
- Auto-restore on new conversation

File Summary

Phase	File	Lines	Purpose
1	<code>src/api/__init__.py</code>	20	Module exports
1	<code>src/api/app.py</code>	100	FastAPI app
1	<code>src/api/models.py</code>	150	Pydantic models
1	<code>src/api/routes/chat.py</code>	150	Chat endpoints
1	<code>src/api/routes/verify.py</code>	100	Verification endpoints
1	<code>src/api/routes/session.py</code>	100	Session endpoints
1	<code>src/api/session_manager.py</code>	150	Session storage
1	<code>tests/test_api.py</code>	200	API tests
2	<code>aeon-web/</code> (Next.js)	~2000	Web UI
3	n8n workflow	~300	Telegram bot

Total: ~3,270 lines

17 Implementation Timeline

Week 1: FastAPI Backend

- **Day 1-2:** Core app, models, session manager
- **Day 3-4:** Chat and verify endpoints
- **Day 5:** Tests, deployment to VPS

Week 2: Next.js Web UI

- **Day 1-2:** Project setup, API client, types
- **Day 3-4:** Chat components, verification display
- **Day 5:** Polish, deploy to Vercel

Week 3: Telegram Integration

- **Day 1-2:** n8n workflow setup
- **Day 3:** Session persistence
- **Day 4-5:** Testing, replace Doctor Feynman



API Specification

Chat Endpoint

POST /api/chat

Content-Type: application/json

Request:

```
{  
  "message": "Is the Hadamard gate unitary?",  
  "session_id": "optional-uuid",  
  "mode": "rigorous" // optional  
}
```

Response:

```
{  
  "response": "Yes, the Hadamard gate is unitary..."
```

```
"session_id": "uuid",
"verified_claims": [
  {
    "claim": "Hadamard is unitary",
    "verified": true,
    "proof": "H†H = I",
    "confidence": 1.0
  }
],
"protocol_state": "ANGEL",
"fidelity": 0.95,
"mode": "rigorous"
}
```

Streaming Chat

```
GET /api/chat/stream?session_id=xxx&message=xxx
Accept: text/event-stream
```

Events:

```
data: {"type": "token", "content": "Yes"}
data: {"type": "token", "content": ", the"}
data: {"type": "verification", "claim": "...", "verified": true}
data: {"type": "done", "session_id": "xxx"}
```

Environment Variables

Backend (.env)

```
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-... # optional
AEON_SESSION_SECRET=random-secret
AEON_CORS_ORIGINS=https://aeon.yourdomain.com
```


Frontend (.env.local)

```
NEXT_PUBLIC_API_URL=https://api.aeon.yourdomain.com
```

Deployment Infrastructure

Domain: bluerose.systems (available in 7 days)

- aeon.bluerose.systems - Web UI (Vercel)
- api.bluerose.systems - FastAPI backend (Hostinger VPS)

Stack:

- **LLM Provider:** Claude (Anthropic) as primary
- **Telegram:** n8n webhook approach
- **Session Storage:** In-memory initially, Redis upgrade path

Success Criteria

Phase 1 Complete When:

- ☐ FastAPI runs on VPS
- ☐ [/api/chat](#) returns verified responses
- ☐ Sessions persist across requests
- ☐ Tests pass

Phase 2 Complete When:

- ☐ Web UI deployed on Vercel
- ☐ Can have verified conversations
- ☐ Mode switching works
- ☐ Responsive design

Phase 3 Complete When:

- ☐ Telegram bot responds as Dolores
 - ☐ Sessions persist per user
 - ☐ Doctor Feynman retired
-

Related Documentation

{{https://www.notion.so/2cee8586e4a2815f9c73ce9bd687a9d2}} - Sprint 4 (LLM Integration foundation)

{{https://www.notion.so/2cee8586e4a28038a713e25058bb6892}} - Sprint 5 (ZK-STARK layer, next after 4.5)

{{https://www.notion.so/2cee8586e4a2812d9b7ddc15d94c80a6}} - Project AEON Command Center